

Actividad guiada: Ejercicio autoinstruccional

Imagina que trabajas como analista de datos en una empresa de retail. El gerente de ventas necesita un reporte para responder preguntas clave sobre los clientes, pedidos y productos. Sin embargo, la información está distribuida en varias tablas de la base de datos (clientes, pedidos, productos y detalle_pedido).

Tu misión es construir consultas SQL en PostgreSQL combinando varias tablas y aplicando operaciones de conjunto, de manera que logres obtener información integrada y útil para la toma de decisiones.

Objetivo de la actividad y materiales necesarios

Objetivos principales

- Aplicar correctamente JOINS (INNER, LEFT, RIGHT, FULL OUTER) en PostgreSQL
- Usar operaciones de conjunto (UNION, INTERSECT, EXCEPT) para resolver necesidades de información
- Representar relaciones entre tablas con base en un modelo entidad-relación
- Elaborar consultas SQL reproducibles y validadas sobre un caso de negocio

Checklist de preparación

- ☒ Acceso a PostgreSQL con un cliente SQL (pgAdmin, DBeaver u otro)
- ☒ Base de datos de ejemplo con las tablas: clientes(id_cliente, nombre, ciudad), pedidos(id_pedido, id_cliente, fecha), productos(id_producto, nombre, precio), detalle_pedido(id_pedido, id_producto, cantidad)
- ☒ Conocimientos previos de SELECT, WHERE, GROUP BY

☐ **Requisitos técnicos:** Conexión activa a PostgreSQL, script SQL de creación y carga mínima de datos (se adjunta en el archivo solución). Tiempo total estimado: 30 minutos.



Instrucciones paso a paso – Parte 1

01

Explora las tablas disponibles (3')

Ejecuta `SELECT * FROM clientes LIMIT 5;` para verificar la tabla.

Micro-check: confirma que ves datos de clientes.

02

Consulta con INNER JOIN (6')

Construye una consulta que muestre todos los pedidos con el nombre del cliente y fecha:

```
SELECT p.id_pedido, c.nombre, p.fecha
FROM pedidos p
INNER JOIN clientes c ON p.id_cliente = c.id_cliente;
```

Micro-check: cada pedido debe tener asociado un cliente.

03

LEFT JOIN para clientes sin pedidos (6')

Identifica qué clientes aún no han comprado:

```
SELECT c.id_cliente, c.nombre
FROM clientes c
LEFT JOIN pedidos p ON c.id_cliente = p.id_cliente
WHERE p.id_pedido IS NULL;
```

Micro-check: la lista debe mostrar solo clientes sin registros en pedidos.

Instrucciones paso a paso – Parte 2

01

Operación de conjunto con INTERSECT (6')

Obtén clientes que compraron en enero y en febrero:

```
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-01-01' AND '2023-01-31'  
INTERSECT  
SELECT id_cliente FROM pedidos WHERE fecha BETWEEN '2023-02-01' AND '2023-02-28';
```

Micro-check: valida que la lista solo incluya IDs presentes en ambos meses.

02

Reporte combinado con agregación (7')

Calcula el total gastado por cliente sumando precio*cantidad:

```
SELECT c.nombre, SUM(dp.cantidad * pr.precio) AS  
total_gastado  
FROM clientes c  
INNER JOIN pedidos p ON c.id_cliente = p.id_cliente  
INNER JOIN detalle_pedido dp ON p.id_pedido = dp.id_pedido  
INNER JOIN productos pr ON dp.id_producto = pr.id_producto  
GROUP BY c.nombre;
```

Micro-check: valida que cada cliente tenga un valor numérico de gasto.

Criterios de éxito y solución completa

✓ Criterios de éxito

El estudiante demuestra:

- Manejo correcto de JOINS y operaciones de conjunto
- Uso de alias y condiciones claras
- Resultados coherentes con las relaciones del modelo

✗ Errores comunes

- Omitir condiciones ON → genera productos cartesianos
- Usar mal la cláusula WHERE en operaciones de conjunto
- No aplicar GROUP BY al usar funciones de agregación

Script de solución SQL

```
-- Creación de tablas de ejemplo
CREATE TABLE clientes (
  id_cliente SERIAL PRIMARY KEY,
  nombre VARCHAR(50),
  ciudad VARCHAR(50)
);

CREATE TABLE pedidos (
  id_pedido SERIAL PRIMARY KEY,
  id_cliente INT REFERENCES clientes(id_cliente),
  fecha DATE
);

CREATE TABLE productos (
  id_producto SERIAL PRIMARY KEY,
  nombre VARCHAR(50),
  precio NUMERIC
);

CREATE TABLE detalle_pedido (
  id_pedido INT REFERENCES pedidos(id_pedido),
  id_producto INT REFERENCES productos(id_producto),
  cantidad INT
);

-- Datos mínimos
INSERT INTO clientes(nombre, ciudad) VALUES
('Ana', 'Santiago'), ('Luis', 'Valparaíso'), ('Marta', 'Santiago');

INSERT INTO productos(nombre, precio) VALUES
('Laptop', 800), ('Mouse', 20), ('Teclado', 50);

INSERT INTO pedidos(id_cliente, fecha) VALUES
(1, '2023-01-10'), (1, '2023-02-15'), (2, '2023-01-20');

INSERT INTO detalle_pedido(id_pedido, id_producto, cantidad)
VALUES
(1, 1, 1), (1, 2, 2), (2, 3, 1), (3, 2, 1);
```